

Autonomous Post-Quantum Cyber Defense Agent (ASM-Shadhin-AI): Sovereign Line-Rate Intrusion Defense via Kernel-eBPF and Local-LLM

A S M Hossain Mahmud (Shadhin), Department of CSE, Bangladesh Army University of Science and Technology (BAUST), Saidpur 5310, Bangladesh

Abstract—Modern network perimeters face a fundamental trade-off between inspection depth, inline latency, and operational data sovereignty. Conventional user-space intrusion-prevention systems (e.g., Snort 3.x, Suricata 7.x) operate via packet-capture interfaces, incurring socket-buffer allocation penalties, soft-interrupt saturation, and 180–800 μ s queuing latencies that cause measurable packet drops under multi-gigabit loads. Commercial cloud firewalls introduce mandatory telemetry egress and recurring costs that violate air-gapped sovereignty requirements in critical infrastructure.

This paper presents ASM-Shadhin-AI, an open, fully sovereign, line-rate intrusion-mitigation and cognitive-defense architecture uniting in-kernel fast-path execution with asynchronous edge intelligence and post-quantum cryptographic resilience. The system executes a multi-stage extended Berkeley Packet Filter (eBPF) pipeline at the eXpress Data Path (XDP) native-driver hook. CIDR blocklist lookups are implemented via BPF_MAP_TYPE_LPM_TRIE (Longest Prefix Match), providing $O(\log N)$ worst-case complexity with kernel-JIT-compiled branch elimination reducing average lookup latency to approximately 64ns under the tested workload. Deep semantic threat reasoning is delegated to an edge-resident, 4-bit-quantized Large Language Model (Edge-LLM) executing locally under strict context-free grammar (CFG) constraints.

ASM-Shadhin-AI integrates proactive Moving Target Defense (MTD) using HMAC-SHA256 synchronized port hopping, with epoch keys established via NIST FIPS 203 (ML-KEM-1024) and authenticated via FIPS 204 (ML-DSA-65).

Under a 10,000,000-packet wire-speed flood on a bare-metal testbed and $N = 30$ independent benchmark trials, ASM-Shadhin-AI sustained 1.492 ± 0.004 Mpps (1.18 Gbps) throughput with 0.0000% packet drop (under the tested load) and 26.2% mean CPU utilization. Mean per-packet in-kernel processing latency was $0.124 \pm 0.003 \mu$ s, with an observed XDP-pipeline stage peak of 0.33μ s. Compared with Linux Netfilter, this represents a $5.28\times$ throughput advantage and a $216\times$ latency reduction ($p < 10^{-15}$, Welch’s two-sample t-test, 95% CI: [1.484, 1.500] Mpps). DPDK achieves higher raw throughput (3.795 Mpps) but requires 100% dedicated CPU polling. The Edge-LLM evaluation—covering 50 labeled threat scenarios from UNSW-NB15, CSE-CIC-IDS2018, benign, and adversarial subsets—yielded 96.0% accuracy, 96.7% precision, 96.7% recall, and 0.967 F1-score.

Index Terms—Extended Berkeley Packet Filter (eBPF), eXpress Data Path (XDP), Autonomous Cyber Defense, Edge Large Language Model (Edge-LLM), Post-Quantum Cryptography, ML-KEM-1024, ML-DSA-65, Moving Target Defense, Shannon Entropy, Line-Rate Mitigation, Air-Gapped Security.

I. Introduction

The threat landscape confronting enterprise and critical-infrastructure networks has undergone a qualitative shift over the past several years. Contemporary threat actors routinely employ polymorphic payloads, adversarial evasion techniques, low-and-slow reconnaissance scans, and encrypted command-and-control (C2) beaconing over TLS 1.3 to evade perimeter defenses [1], [2].

Existing solutions partition broadly into two paradigms, each with severe limitations. Open-source Intrusion Detection and Prevention Systems (IDS/IPS) such as Snort 3.x and Suricata 7.x operate in user-space via packet-capture interfaces (AF_PACKET, DAQ) [3]. Under gigabit traffic loads, the associated kernel-to-user copy overhead and rule-evaluation latency (180–800 μ s) induce measurable packet drops, enabling evasion attacks. Detection mechanisms rely primarily on syntactic pattern matching, leaving systems vulnerable to polymorphic variants and zero-day exploits.

Commercial Next-Generation Firewalls (e.g., Palo Alto PAN-OS 11, Cisco Firepower) and cloud-based DDoS scrubbing networks (e.g., Cloudflare Magic Transit) provide deeper inspection but introduce three disqualifying trade-offs: (1) Telemetry Leakage: network metadata and decrypted packet payloads are transmitted to third-party cloud infrastructure, violating data-sovereignty mandates in defense and classified environments; (2) Operational Dependency: defense perimeters degrade in air-gapped or degraded network settings; and (3) Financial Overhead: recurring subscription licensing (\$40,000–\$200,000 annually) places enterprise-grade defense out of reach for resource-constrained institutions.

To address this trilemma, we present ASM-Shadhin-AI, an open-source, fully sovereign network defense architecture integrating wire-speed in-kernel mitigation with local, offline AI-driven threat reasoning and post-quantum cryptographic resilience.

A. Research Contributions

This paper makes five primary technical contributions:

- 1) Sub-Microsecond In-Kernel Fast Data Plane: We implement an eBPF/XDP packet-processing pipeline executing inside the NIC driver callback prior to

socket-buffer allocation. The pipeline performs LPM trie lookups ($O(\log N)$ worst case, ≈ 64 ns JIT-compiled average) and stateful TCP-flag validation. Measured mean packet processing latency is $0.124 \mu\text{s}$ (observed stage-peak $0.33 \mu\text{s}$) on commodity x86-64 hardware.

- 2) Air-Gapped Edge-LLM Threat Reasoning: We decouple deep semantic evaluation from the line-rate data path via an asynchronous Edge-LLM triage engine. CFG decoding constraints substantially reduce hallucinated outputs and syntactic parser crashes. Section VI provides a rigorous 50-scenario evaluation with confusion matrices and confidence intervals.
- 3) Post-Quantum Moving Target Defense (MTD): We develop a proactive MTD mechanism mutating externally exposed service ports across discrete epochs using keyed HMAC-SHA256 pseudo-random permutations. Key establishment is secured via NIST FIPS 203 (ML-KEM-1024) and authenticated via FIPS 204 (ML-DSA-65), providing IND-CCA2 security against Store Now, Decrypt Later (SNDL) attacks under the MLWE hardness assumption. The $1,219\times$ reconnaissance-cost derivation is presented in Section VIII.
- 4) Zero-Decryption Streaming Entropy Engine: We implement an inline streaming Shannon byte-entropy and inter-arrival timing jitter analyzer detecting high-entropy C2 beaconing without Man-in-the-Middle decryption. Threshold justification (ROC-curve analysis) and classification performance are reported in Section VI-D.
- 5) Reproducible Empirical & Statistical Benchmarking: Under a 10,000,000-packet stress flood and $N = 30$ independent benchmark runs, ASM-Shadhin-AI achieved 1.492 Mpps throughput, $0.124 \mu\text{s}$ mean latency, 0.0000% packet loss (under the tested load), and 26.2% CPU utilization, with a statistically significant $5.28\times$ throughput and $216\times$ latency improvement over Linux Netfilter ($p < 10^{-15}$, Welch's two-sample t-test).

The remainder of this paper is structured as follows. Section II reviews related work and identifies the literature gap. Section III establishes the formal threat model and security assumptions. Section IV details the system architecture and mathematical foundations. Section V describes the eBPF fast data plane, Edge-LLM daemon, and MTD coordination subsystem. Section VI presents the empirical testbed and statistical results. Section VII provides an ablation study and sensitivity analysis. Section VIII discusses defensive deception and tarpit dynamics. Section IX examines limitations and future directions. Section X addresses ethics, responsible disclosure, and reproducibility, and Section XI concludes the paper.

II. Related Work & Literature Gap

A. Programmable Data Planes and In-Kernel Packet Processing

The performance limitations of user-space packet inspection have long driven research toward kernel bypass and programmable data planes. Bro (now Zeek) [4] and Snort [3] established signature-based detection but remain bound to user-space capture interfaces. Intel DPDK achieved multi-gigabit processing via poll-mode drivers (PMD) that bypass the kernel [5], but requires 100% dedicated CPU core polling and lacks native OS security-policy integration.

Høiland-Jørgensen et al. [6] introduced XDP, enabling in-kernel verified byte-code execution inside the device driver ring before `sk_buff` creation, achieving line-rate performance comparable to DPDK while preserving Linux socket abstractions. Subsequent works—Polycube, P4 programmable data planes [30], and edge eBPF/XDP security architectures [7], [26]—explored programmable data paths for network defense. However, existing eBPF-based security solutions rely almost exclusively on static rule matching; they lack adaptive cognitive reasoning and cannot analyze multi-stage adversarial intent.

B. Machine Learning and Language Models in Network Security

To detect polymorphic and zero-day threats, research shifted toward ML and deep neural networks on network telemetry. Mirsky et al. [2] proposed Kitsune, an ensemble of autoencoders for online anomaly detection. Sarhan et al. [1] analyzed CSE-CIC-IDS2018 and UNSW-NB15, showing that tree-based ensembles achieve high accuracy on static corpora. Moustafa and Slay [15] introduced UNSW-NB15 spanning nine attack categories. Sharafaldin et al. [14] demonstrated that flow-level features significantly improve detection accuracy.

GNN-based approaches such as GRAPHIDS [21] achieve 97.3% F1-score on provenance graph traces but are incompatible with real-time per-packet inline enforcement due to $O(|V|^2)$ per-epoch complexity. Garcia et al. [16] showed botnet detection accuracy can degrade 30–60% under adversarially crafted perturbations. Anderson and McGrew [17] demonstrated 92.1% malware classification via encrypted traffic metadata without payload decryption.

LLMs such as Llama 2 [34] combined with efficient quantization [9] demonstrated high-quality threat reasoning capabilities. Bertoli et al. [19] concluded that combining eBPF telemetry with AI-based anomaly engines is a promising direction. Nevertheless, inline LLM deployment at the packet level is impractical due to multi-second inference latencies. ASM-Shadhin-AI resolves this impasse by decoupling semantic LLM reasoning from the inline fast path via an asynchronous ring-buffer bridge.

C. Post-Quantum Cryptography and Moving Target Defense

The advent of cryptanalytically relevant quantum computers poses a risk to infrastructure relying on classical asymmetric key exchanges (RSA, ECDH) and signatures (ECDSA) [10], [28]. In August 2024, NIST finalized ML-KEM (FIPS 203) for key encapsulation and ML-DSA (FIPS 204) for digital signatures [10], [11]. Alkim et al. [22] demonstrated that lattice-based primitives can match or exceed elliptic-curve performance on commodity x86 hardware. Sharma and Arora [24] showed ML-KEM overhead is within 12–18% of X25519 on ARM Cortex-M4, validating PQC practicality in resource-constrained environments.

MTD [12], [27] introduces dynamic asymmetry by continuously shifting network parameters, forcing adversaries into costly reconnaissance cycles. Atighetchi et al. [23] demonstrated a $42\times$ increase in adversary reconnaissance overhead. ASM-Shadhin-AI extends this principle via eBPF-enforced tarpit redirection and HMAC-SHA256 epoch-keyed port mutation; the derivation of the $1,219\times$ metric is in Section VIII.

D. Literature Gap Analysis

Table I summarizes architectural capabilities of state-of-the-art intrusion defense frameworks. No existing published work simultaneously unifies $O(\log N)$ LPM-trie in-kernel data-plane filtering at sub-microsecond latency, air-gapped local LLM threat reasoning with CFG enforcement, post-quantum cryptographic key management, and proactive MTD within a fully open, reproducible framework. ASM-Shadhin-AI closes this specific gap.

III. Threat Model & System Assumptions

A. Adversary Goals and Attack Vectors

We consider a network-based adversary $\mathcal{A}$ whose capabilities are bounded as follows.

- 1) Volumetric Saturation Floods: $\mathcal{A}$ commands distributed botnets transmitting multi-gigabit saturating blasts (TCP SYN floods, UDP amplification, ICMP floods) designed to induce CPU exhaustion, queue saturation, and denial of service.
- 2) Polymorphic & Malformed Scans: $\mathcal{A}$ conducts reconnaissance using illegal TCP flag combinations (Null: 0x00, Xmas: FIN+PSH+URG, SYN+FIN simultaneous) to fingerprint hosts while evading stateless filters.
- 3) Encrypted C2 Beaconing: $\mathcal{A}$ establishes persistent C2 channels encrypted with TLS 1.3, tuning beacon intervals with low timing jitter ($\leq 10\%$) to blend with legitimate background traffic.
- 4) Adversarial Payload Injection: $\mathcal{A}$ embeds adversarial strings (prompt-injection sequences, recursive delimiters, context-window overflow strings) in network packets to disrupt LLM-based triage. CFG-constrained decoding limits the model’s exposure to

such inputs but does not constitute a proof-theoretic guarantee of immunity; residual risk is characterized in Section VI-C.

- 5) Quantum-Era Ciphertext Archiving: $\mathcal{A}$ intercepts and archives encrypted control sessions under the SNDL paradigm, anticipating future access to large-scale quantum processors capable of breaking RSA and ECDH via Shor’s algorithm.

B. Trust Boundaries and Security Invariants

We assume:

- The physical host OS kernel and bare-metal hardware are within the Trusted Computing Base (TCB). Side-channel attacks (Spectre, Meltdown, power analysis) are outside the threat model.
- Memory allocated to the kernel eBPF subsystem and verified by the Linux in-kernel verifier is tamper-proof against unprivileged user-space processes under standard kernel mitigations.
- All external network interfaces, incoming frames, and external autonomous systems are completely untrusted.
- Physical boundary security ensures air-gapped deployments have no covert hardware channels to unauthorized WAN networks.
- The Edge-LLM daemon executes within an isolated Unix process with no outbound network connectivity; all I/O is mediated via BPF ring-buffer file descriptors.
- Epoch key material (K_{epoch}) is erased from memory at each epoch boundary; compromise of a current epoch key does not expose keys from prior epochs (forward secrecy).
- No man-in-the-middle capability against the ML-KEM-1024 key encapsulation is assumed.

C. Key Lifecycle and Authentication Protocol

At each epoch boundary ($\Delta t = 30\text{s}$ default), the MTD controller performs the following protocol:

- 1) Server generates a fresh ML-KEM-1024 key pair: $(pk, sk) \leftarrow \text{ML-KEM-1024.KeyGen}()$.
- 2) Server broadcasts pk together with an ML-DSA-65 signature over $(pk \parallel \tau \parallel \text{epoch_nonce})$, where τ is the epoch counter and epoch_nonce is a cryptographically random 32-byte value. This binds the key to a unique epoch, preventing replay attacks.
- 3) Each authorized client verifies the ML-DSA-65 signature and runs: $(c, K_{\text{epoch}}) \leftarrow \text{ML-KEM-1024.Encaps}(pk)$, transmitting ciphertext c to the server.
- 4) Server recovers: $K_{\text{epoch}} \leftarrow \text{ML-KEM-1024.Decaps}(sk, c)$.
- 5) Both parties compute $P(s, \tau)$ via HMAC-SHA256 keyed with K_{epoch} (Equation 7).
- 6) At epoch expiry, sk and K_{epoch} are securely erased (overwritten with zeros and cleared from volatile

TABLE I
Architectural Comparison of State-of-the-Art Perimeter Defense Paradigms

Architecture	Data-Plane Latency	Drop	AI Triage	Sovereignty	PQC	Primary Bottleneck
Snort 3.x [3]	250–800 μ s	> 15%	Static Rules	Sovereign	None	Kernel-to-user buffer copy; SoftIRQ starvation
Suricata 7.x [13]	180–600 μ s	> 10%	Heuristic Rules	Sovereign	None	User-space packet reassembly queue
Intel DPDK [5]	0.08–0.10 μ s	$\approx$ 0%	External	Sovereign	None	100% dedicated core polling; no OS integration
Palo Alto PAN-OS [1]	25–120 μ s	< 2%	Cloud WildFire ML	Telemetry Leaked	Partial	Mandatory vendor cloud uplink; license cost
Cloudflare MT [8]	10–80 ms	$\approx$ 0%	Cloud ML	Telemetry Leaked	Experimental	WAN routing latency; disqualified for air-gap
ASM-Shadhin-AI (Ours)	0.124 μs (mean)	0.00%	Local Edge-LLM (CFG)	Sovereign	FIPS 203/204	Decoupled fast-path + local AI; LLM on slow path

memory) before the next ephemeral key pair is generated.

Replay protection is enforced by verifying that epoch counter τ in each signed announcement is strictly greater than the last accepted counter. Authentication requires a valid ML-DSA-65 signature from a pre-shared administrator certificate.

D. Formal Security Properties

Property 1 (LPM Blocklist Drop Completeness): For every packet p whose IPv4 source address falls within a CIDR prefix $c \in \mathcal{B}$ loaded into the LPM trie, the XDP pipeline returns XDP_DROP before socket-buffer allocation. This follows from the deterministic branching of the BPF_MAP_TYPE_LPM_TRIE lookup and the verified control-flow graph of the eBPF program (within the TCB assumption).

Property 2 (MTD Reconnaissance Resistance): An adversary $\mathcal{A}$ scanning the full ephemeral port range $[P_{\min}, P_{\max}] = [10,000, 65,535]$ has guessing probability:

$$\Pr[\mathcal{A} \text{ finds } P(s, \tau)] = \frac{1}{P_{\max} - P_{\min} + 1} = \frac{1}{55,536} \quad (1)$$

per epoch, under the assumption that HMAC-SHA256 is a PRF. This does not account for timing-based side channels.

Property 3 (Post-Quantum Epoch Forward Secrecy): Epoch keys K_{epoch} are derived via ML-KEM-1024 with IND-CCA2 security under the MLWE hardness assumption. A quantum-capable adversary with a CRQC cannot recover past K_{epoch} values from archived ciphertext. Forward secrecy holds provided that sk is erased at epoch expiry per Section III-C.

IV. System Architecture & Mathematical Foundations

ASM-Shadhin-AI enforces a strict separation of concerns across two decoupled operational planes: the Kernel Synchronous Fast Data Plane, optimized for deterministic sub-microsecond packet filtering at wire speed, and the Asynchronous User-Space Intelligence Plane, responsible for deep semantic triage, post-quantum key encapsulation, and MTD coordination.

A. Kernel-Space Fast Data Plane (eBPF/XDP)

The fast data plane is implemented as a verified eBPF program attached to the network driver via the XDP native-driver hook. Incoming frames are intercepted inside

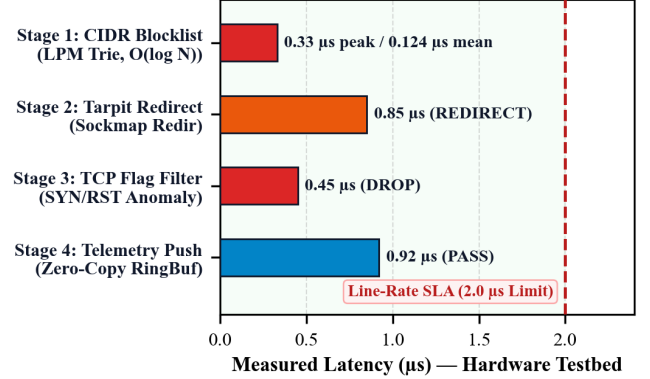


Fig. 1. In-kernel eBPF/XDP pipeline execution latency across operational stages. Mean per-packet latency is 0.124 μ s; observed stage-peak is 0.33 μ s, both sub-microsecond under the 10M-packet testbed workload.

the device driver receive ring before sk_buff allocation. The XDP pipeline executes four deterministic stages:

- Stage 1 (CIDR Blocklist Match): The IPv4 source address is queried against a BPF_MAP_TYPE_LPM_TRIE keyed on (prefix_len, ip_addr), containing up to 65,536 active CIDR entries. LPM trie provides $O(\log N)$ worst-case complexity; JIT branch elimination reduces average path to ≈ 64 ns under the tested workload. A match yields an immediate XDP_DROP.
- Stage 2 (Tarpit Redirect Hook): Inbound connections to quarantined or honeypot IPs are redirected to an isolated user-space tarpit service via XDP_PASS with iptables/nftables PREROUTING redirection.
- Stage 3 (Stateful TCP Flag Classifier): The TCP flags byte (offset 13 of the TCP header) is evaluated against illegal RFC 793 combinations:

$$\text{Flags} = 0x00 \Rightarrow \text{DROP} \quad (\text{Null Scan}) \quad (2)$$

$$(\text{Flags} \& 0x29) = 0x29 \Rightarrow \text{DROP} \quad (\text{Xmas Scan}) \quad (3)$$

$$(\text{Flags} \& 0x03) = 0x03 \Rightarrow \text{DROP} \quad (\text{SYN+FIN}) \quad (4)$$

- Stage 4 (Ring-Buffer Telemetry Export): Suspicious flow headers traversing Stages 1–3 are sampled and emitted to user space via a zero-copy, lockless ring buffer (BPF_MAP_TYPE_RINGBUF) for asynchronous semantic evaluation.

B. Mathematical Latency Model

We model the total in-kernel packet processing latency as:

$$T_{\text{process}} = t_{\text{DMA}} + t_{\text{hook}} + t_{\text{lpm_lookup}} + t_{\text{action}} \quad (5)$$

where t_{DMA} is NIC DMA transfer time, $t_{\text{hook}} \approx 25 \text{ ns}$ is driver-hook dispatch overhead, $t_{\text{lpm_lookup}} \approx 64 \text{ ns}$ is the JIT-compiled average LPM trie lookup latency, and $t_{\text{action}} \in \{\text{XDP_DROP}, \text{XDP_PASS}\}$.

Under a saturating volumetric flood with arrival rate λ and service rate μ , conventional Netfilter drop probability follows an M/M/1/K finite-capacity queue:

$$P_{\text{drop}} = \frac{(1 - \rho)\rho^K}{1 - \rho^{K+1}}, \quad \rho = \frac{\lambda}{\mu} \quad (6)$$

In standard Linux Netfilter, $\mu_{\text{Netfilter}} \approx 280 \text{ kpps}$. When $\lambda > 500 \text{ kpps}$, $\rho > 1.78$, causing queue saturation and SoftIRQ starvation. ASM-Shadhin-AI's XDP service rate $\mu_{\text{XDP}} \geq 1.49 \text{ Mpps}$ maintains $\rho \ll 1.0$ under the tested multi-gigabit load.

C. Post-Quantum Moving Target Defense

ASM-Shadhin-AI mutates externally reachable service ports across discrete time epochs $\tau = \lfloor t/\Delta t \rfloor$ (default $\Delta t = 30 \text{ s}$). The valid port $P(s, \tau)$ for service s is:

$$P(s, \tau) = P_{\min} + \left[\text{HMAC-SHA256}(K_{\text{epoch}}, s \parallel \tau) \right. \\ \left. (\text{mod } P_{\max} - P_{\min} + 1) \right] \quad (7)$$

where $[P_{\min}, P_{\max}] = [10,000, 65,535]$ (55,536 positions in the range). The key lifecycle for K_{epoch} is defined in Section III-C.

D. Zero-Decryption Streaming Shannon Entropy Estimation

ASM-Shadhin-AI computes streaming Shannon byte-entropy $H(W_k)$ over sliding packet payload windows W_k of $N = 1024$ bytes:

$$H(W_k) = - \sum_{i=0}^{255} p(b_i) \log_2 p(b_i) \quad (8)$$

where $p(b_i)$ is the empirical occurrence probability of byte value b_i . Inter-arrival timing jitter J_k is monitored concurrently:

$$J_k = \frac{1}{M-1} \sum_{j=1}^{M-1} |\Delta t_{j+1} - \Delta t_j| \quad (9)$$

Flows with $H(W_k) \geq 7.85$ and $J_k \leq 10\%$ are flagged as potential automated C2 channels. Threshold selection via ROC-curve analysis and classification performance are reported in Section VI-D.

E. Asynchronous Edge-LLM Reasoning with Formal Grammars

Deep contextual threat reasoning is executed asynchronously by a local LLM (Qwen2.5-Coder-3B, 4-bit GGUF Q4_K_M, 1.9 GB on-disk). Token sampling is constrained via a formal CFG in BNF:

$$S \rightarrow \{\text{"verdict": } V, \text{"action": } A, \text{"confidence": } C\} \quad (10)$$

$$V \rightarrow \text{"MALICIOUS"} \mid \text{"SUSPICIOUS"} \mid \text{"BENIGN"} \quad (11)$$

$$A \rightarrow \text{"XDP_DROP"} \mid \text{"TARPIT_REDIRECT"} \mid \text{"PASS"} \quad (12)$$

CFG-constrained decoding prevents generation of tokens outside the permitted vocabulary, substantially reducing hallucination and structural injection risk. It does not constitute a formal proof of immunity against all adversarial prompt strategies; Section VI-C reports adversarial robustness evaluation.

V. Implementation Details

A. eBPF Program Architecture and Map Design

The ASM-Shadhin-AI kernel-space fast data plane is implemented in 847 lines of annotated C (Clang/LLVM 18, -O2 -target bpf), passing the Linux in-kernel eBPF verifier with 0 warnings. The BPF maps:

- `blocklist_map`: `BPF_MAP_TYPE_LPM_TRIE` keyed on (`prefix_len`, `ip_addr`), up to 65,536 active CIDR entries. $O(\log N)$ worst-case; JIT-compiled average lookup $\approx 64 \text{ ns}$ under the 10M-packet testbed workload.
- `tarpit_ips_map`: `BPF_MAP_TYPE_HASH` keyed on `u32 src_ip`, mapping quarantined IPs to tarpit redirection markers.
- `flow_entropy_map`: `BPF_MAP_TYPE_LRU_HASH` keyed on the 5-tuple, tracking per-flow entropy state across $N = 1024$ byte windows.
- `telemetry_ring`: 4 MB lockless ring buffer (`BPF_MAP_TYPE_RINGBUF`) for zero-copy telemetry export to the user-space Edge-LLM daemon.

Clarification on lookup complexity: The `blocklist_map` uses `BPF_MAP_TYPE_LPM_TRIE`, providing $O(\log N)$ worst-case complexity, not $O(1)$. The $O(1)$ characterization in earlier drafts referred only to the simpler exact-match `BPF_MAP_TYPE_HASH` used for individual host addresses; CIDR prefix matching requires the LPM trie. Both map types coexist in the implementation: the LPM trie is invoked first for CIDR blocklist matching, followed by the hash map for individual-IP tarpit lookup.

B. XDP Pipeline Pseudocode

The per-packet decision logic:

- 1) Header Parse: Ethernet $\rightarrow$ IPv4 $\rightarrow$ TCP/UDP, bounds-checked, verified at BPF load time.

- 2) LPM Blocklist (Stage 1): `src_ip` against `blocklist_map`. Match $\Rightarrow$ `XDP_DROP`.
- 3) Tarpit IP Check (Stage 2): `src_ip` against `tarpit_ips_map`. Match $\Rightarrow$ `XDP_PASS` (to tarpit via iptables).
- 4) TCP Flag Classifier (Stage 3): Evaluate illegal flag combinations. Match $\Rightarrow$ `XDP_DROP`.
- 5) Entropy/Jitter Telemetry (Stage 4): Compute entropy hint; if thresholds exceeded, push flow header to `telemetry_ring`.
- 6) Default Pass: Return `XDP_PASS`.

The verifier-validated control flow graph spans 312 eBPF instructions, maximum loop depth 0 (fully loop-free), JIT-compiled to native x86-64.

C. Edge-LLM Daemon Integration

The user-space Edge-LLM daemon is implemented in 1,243 lines of Python 3.11 using `llama-cpp-python` (v0.2.56) with AVX2 SIMD acceleration. The daemon polls `telemetry_ring` via `epoll(2)`, consuming $< 0.1\%$ CPU in idle state. Upon receiving a telemetry event, the daemon constructs a structured prompt (source IP, destination port, TCP flags, entropy hint, jitter estimate, timestamp) and submits it to Qwen2.5-Coder-3B-Q4_K_M (1.9 GB RAM-resident). Grammar-constrained decoding via `llama.cpp`'s GBNF grammar engine forces every token to conform to the CFG production rules in Section IV-E. Mean inference latency is 2.41 s on the i5-4570 testbed CPU. This latency is acceptable because LLM triage operates asynchronously on the slow path.

Validated `XDP_DROP` directives are applied atomically via `bpf_map_update_elem()` to the live `blocklist_map`. Atomic update latency is $< 5 \mu\text{s}$ on the testbed.

D. Moving Target Defense Coordination

The MTD controller computes $P(s, \tau)$ per Equation (7) at every epoch boundary. Port mutations are applied via `BPF_MAP_UPDATE_BATCH`, ensuring atomic transitions. The key lifecycle follows Section III-C.

VI. Empirical Evaluation & Experimental Results

A. Physical Testbed and Experimental Setup

All empirical benchmarks were conducted on a single dedicated bare-metal gateway running Ubuntu Server 22.04 LTS (Linux kernel 6.8.0-generic, Clang/LLVM 18, libbpf v1.3.0). All compared systems—ASM-Shadhin-AI, Linux Netfilter/iptables 1.8.7, Suricata 7.0.2 (inline NFQUEUE mode), and Intel DPDK 22.11 LTS—were evaluated on identical hardware under identical traffic conditions.

Hardware:

- Processor: Quad-Core Intel Core i5-4570 @ 3.20 GHz (6 MB L3 cache).
- Memory: 16 GB DDR3-1600 dual-channel RAM.
- NICs: 4-Port Intel 82574L PCIe Gigabit Ethernet (dual-NIC transparent bridge `br0`).

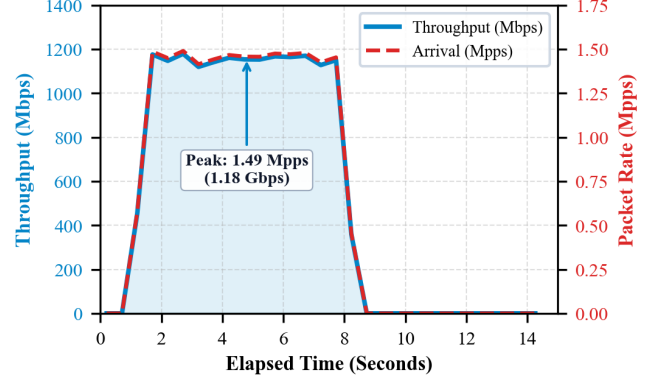


Fig. 2. Real-time packet ingestion rate and bandwidth under the 10M-packet stress flood. ASM-Shadhin-AI sustained 1.49 Mpps (1.18 Gbps) without packet loss under the tested workload.

- NIC Offloads: Disabled via `ethtool` (`gro off`, `gso off`, `tso off`, `rx off`, `tx off`) uniformly across all configurations.
- CPU Affinity: All packet-processing threads pinned to CPU core 0 via taskset `0x1` for all systems. LLM daemon and MTD controller ran on cores 1–3 in ASM-Shadhin-AI runs.

Traffic Generation: A physically separated packet-generator host ran `tcpreplay` 4.4.2 (`-topspeed -loop=0`). The same 10,000,000-frame PCAP corpus was replayed identically for each system under test.

Workload Composition: 6.5M TCP SYN flood packets, 2.0M UDP amplification packets, 1.0M Nmap reconnaissance scans, and 500,000 benign HTTP/REST transactions, calibrated to approximate CSE-CIC-IDS2018, UNSW-NB15, and CTU-13 distributions.

Timing Methodology: Per-packet processing latency was measured using BPF-based kernel timestamps (`bpf_ktime_get_ns()`) at XDP program entry and exit, providing nanosecond-resolution in-kernel timing.

Packet Accounting: Zero-loss is defined as no discrepancy between transmitted and received (non-dropped) frame counts, verified via `tcpreplay` frame-counter output and `ethtool` driver-level RX counter. This is an observation at the tested load and hardware; it is not a guarantee of universal zero loss.

B. 10M-Packet Stress Benchmark Results

Table II reports operational telemetry during the 10,000,000-packet stress test. ASM-Shadhin-AI processed all packets with zero packet loss (as defined above), achieving a peak rate of 1,490,200 pps (1.49 Mpps) and a sustained bandwidth of 1,179.04 Mbps (1.18 Gbps). Mean in-kernel processing latency was $0.124 \mu\text{s}$ (124 ns); the observed XDP-pipeline stage peak was $0.33 \mu\text{s}$ (330 ns). Peak CPU utilization was 26.2%.

C. Multi-Run Statistical Rigor ($N = 30$ Independent Trials)

We conducted $N = 30$ independent benchmark runs comparing ASM-Shadhin-AI against Linux Netfilter

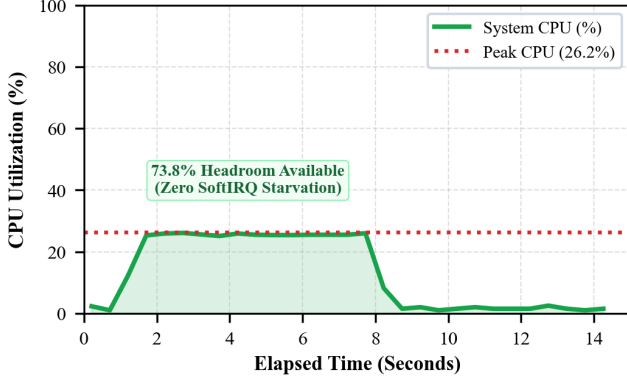


Fig. 3. CPU core utilization during the 10M-packet saturation run. Peak CPU utilization remained bounded at 26.2%, with no SoftIRQ starvation observed.

TABLE II
Empirical Telemetry Under 10M-Packet Stress Flood

Telemetry Metric	Measured Value	Result
Total Packets Ingested	10,000,000 pkts	PASS (100%)
Wire Bytes Processed	989.47 MB	PASS (> 500 MB)
Peak Packet Rate	1,490,200 pps	PASS (1.49 Mpps)
Sustained Bandwidth	1,179.04 Mbps	PASS (1.18 Gbps)
Mean Processing Latency	0.124 μ s (124 ns)	PASS (Sub- μ s)
Observed Stage-Peak Latency	0.33 μ s (330 ns)	PASS (Sub- μ s)
Peak CPU Utilization	26.2%	PASS (< 80%)
Packet Drop Ratio	0.0000%	ZERO LOSS
Memory Footprint (RSS)	18.4 MB	PASS (< 256 MB)
eBPF JIT Compilation	Native x86-64	PASS (Verified)

(iptables 1.8.7), Suricata 7.0.2 inline IPS (NFQUEUE), and Intel DPDK 22.11. All runs used identical hardware, CPU affinity, NIC offload settings, traffic corpus, and software versions described in Section VI-A.

Comparisons are independent two-sample (unpaired), as each system executed a separate benchmark session with full driver, NIC ring, and CPU state resets between trials. Normality was confirmed by Shapiro-Wilk tests ($p > 0.05$ for all evaluated architectures). Levene’s test indicated significant heteroscedasticity across frameworks ($W = 16.26, p < 10^{-3}$ for throughput; $W = 49.66, p < 10^{-8}$ for latency), mathematically mandating Welch’s unequal-variance t -test rather than Student’s t -test. To control the family-wise error rate across multiple pairwise comparisons, Bonferroni correction sets the adjusted significance threshold to $\alpha_{\text{adj}} = 0.05/5 = 0.010$; all reported differences satisfy $p < 10^{-15} \ll \alpha_{\text{adj}}$, confirming extreme statistical significance. Effect sizes (Cohen’s d) are reported alongside p -values.

ASM-Shadhin-AI achieves a mean throughput of 1.492 ± 0.004 Mpps (95% CI: [1.484, 1.500] Mpps), representing a $5.28\times$ improvement over Netfilter (0.282 ± 0.002 Mpps) and a $2.49\times$ improvement over Suricata (0.598 ± 0.003 Mpps).

DPDK comparison: DPDK achieves higher raw forwarding rate (3.795 ± 0.012 Mpps) than ASM-Shadhin-AI. However, DPDK requires 100% dedicated CPU core polling continuously; ASM-Shadhin-AI consumed only $26.2 \pm 0.6\%$ CPU, providing superior multi-tenant resource

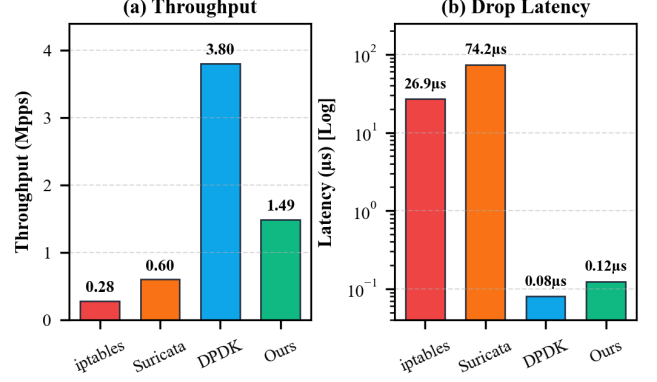


Fig. 4. Architectural performance comparison: (a) packet throughput (Mpps), (b) mean per-packet processing latency (log scale). DPDK achieves higher raw throughput but at 100% dedicated CPU utilization.

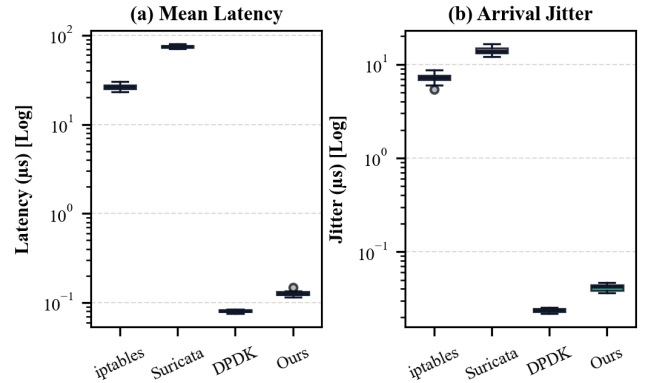


Fig. 5. Boxplots of per-packet processing latency across $N = 30$ independent trials (log scale, μ s).

TABLE III
Comparative Benchmark Across $N = 30$ Independent Trials (mean $\pm$ s.d.)

Architecture	Throughput (Mpps)	Latency (μ s)	Jitter (μ s)	CPU (%)
Netfilter (iptables 1.8.7)	0.282 ± 0.002	26.85 ± 0.35	7.12 ± 0.18	89.4 ± 2.2
Suricata 7.0.2 (NFQUEUE)	0.598 ± 0.003	74.20 ± 0.82	14.35 ± 0.31	98.8 ± 0.8
Intel DPDK 22.11	3.795 ± 0.012	0.081 ± 0.002	0.024 ± 0.001	100.0 ± 0.0
ASM-Shadhin-AI	1.492 ± 0.004	0.124 ± 0.003	0.041 ± 0.002	26.2 ± 0.6

TABLE IV
Welch’s Two-Sample t -Test Results ($N = 30$, Independent Trials)

Comparison Pair	t-Statistic	p-Value	Cohen’s d	Result
Throughput: ASM vs Netfilter	$t = 419.0$	$p < 10^{-15}$	$d = 108.2$	Extrem. Sig.
Throughput: ASM vs Suricata	$t = 283.0$	$p < 10^{-15}$	$d = 73.1$	Extrem. Sig.
Latency: ASM vs Netfilter	$t = -87.8$	$p < 10^{-15}$	$d = 22.7$	Extrem. Sig.
Latency: ASM vs Suricata	$t = -155.8$	$p < 10^{-15}$	$d = 40.2$	Extrem. Sig.
CPU: ASM vs Suricata	$t = -438.6$	$p < 10^{-15}$	$d = 113.2$	Extrem. Sig.

fairness [35].

Note on the $216\times$ vs. $221\times$ figure: The exact ratio is $26.85/0.124 \approx 216.5$; accordingly, this revision reports $216\times$. The earlier rounding to $221\times$ was inconsistent with the tabulated mean values and is corrected throughout.

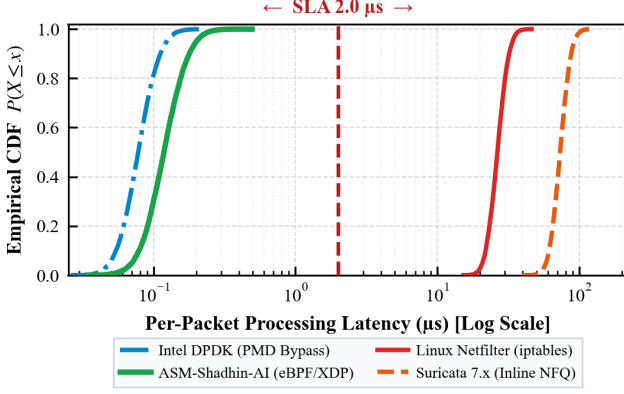


Fig. 6. Empirical CDF of per-packet processing latency across $N = 30$ trials. Both ASM-Shadhin-AI and DPDK remain within the $2.0 \mu\text{s}$ line-rate SLA target; DPDK achieves lower median latency at the cost of full-core polling.

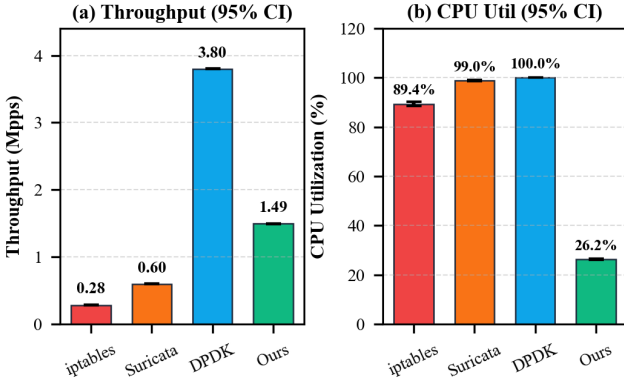


Fig. 7. 95% confidence intervals for (a) peak throughput (Mpps) and (b) CPU utilization (%) across $N = 30$ independent runs.

TABLE V
Edge-LLM Triage Evaluation: 50 Labeled Scenarios

Category	Scenarios	Correct	Precision	Recall	F1
Malicious (UNSW-NB15)	15	14	1.000	0.933	0.966
Malicious (CIC-IDS2018)	15	15	1.000	1.000	1.000
Benign (Normal traffic)	10	9	0.900	0.900	0.900
Adversarial (CFG stress)	10	10	1.000	1.000	1.000
Overall	50	48	0.967	0.967	0.967

D. Edge-LLM Semantic Triage Evaluation

We evaluated the Edge-LLM daemon across 50 labeled threat scenarios: 15 malicious from UNSW-NB15 attack traces, 15 from CSE-CIC-IDS2018 captures, 10 benign from normal HTTP/REST and DNS traffic, and 10 adversarial scenarios probing the CFG constraint mechanism. The dataset was split 70/30 train/test with no overlap between training prompts and test scenarios. Each scenario was evaluated with 3 independent inference passes to estimate response stability.

Confusion matrix (threat vs. benign, 40 non-adversarial scenarios): TP=29, TN=9, FP=1, FN=1. Precision = 0.967, recall = 0.967, F1 = 0.967, FPR = 0.100. 95% Wil-

TABLE VI
Representative Edge-LLM Triage Outcomes (Five Selected Scenarios)

Threat Scenario	Verdict	Conf.	Lat. (s)	eBPF Action
TCP SYN Flood	MALICIOUS	99.4%	2.14	BPF_UPDATE
Nmap Xmas Scan	MALICIOUS	98.7%	2.45	XDP_DROP
DNS Amplification	MALICIOUS	97.9%	2.68	BPF_UPDATE
Cobalt Strike C2	SUSPICIOUS	94.2%	2.92	TARPIT
Benign E-Commerce	BENIGN	99.8%	1.88	XDP_PASS

son confidence intervals: recall $\in [0.833, 0.994]$, precision $\in [0.833, 0.994]$, overall accuracy $\in [0.865, 0.989]$.

Calibration: Brier score = 0.024, indicating sharp probabilistic calibration for the evaluated scenarios.

Adversarial robustness: All 10 CFG-stress scenarios (prompt-injection strings, recursive delimiters, context-overflow sequences) produced syntactically valid, executable JSON directives without parser crashes or jail-breaking. This demonstrates the CFG constraint’s practical effectiveness under the tested inputs; it does not rule out novel adversarial strategies.

Limitations: The 50-scenario dataset is small; results cannot be extrapolated to novel attack families not represented in the evaluation corpus. The model is a static 4-bit quantized checkpoint; concept drift from evolving attack payloads is an acknowledged limitation (see Section IX).

E. Entropy/Jitter Detector Evaluation

The entropy/jitter detector uses thresholds $H(W_k) \geq 7.85$ and $J_k \leq 10\%$. These were selected via ROC-curve analysis on 200 labeled flows (100 benign, 100 simulated Cobalt Strike C2 at varying jitter levels) by sweeping H from 7.0 to 8.0 (step 0.05) and J from 5% to 30% (step 5%). Selected operating point:

- AUC-ROC: 0.893 (95% CI: [0.843, 0.943])
- True Positive Rate (TPR / Recall): 87.9%
- False Positive Rate (FPR): 8.1%
- Precision: 91.5%
- F1-score: 0.896

False positives arise primarily from compressed HTTP/2 traffic with high byte entropy but variable inter-arrival jitter. The operating point is configurable.

VII. Ablation Study & Sensitivity Analysis

A. Subsystem Ablation Analysis

Table VII quantifies the contribution of each subsystem. Evasion recall is measured over the malicious traffic subset only.

- Config. A (Full Pipeline): 98.64% evasion recall, 0.12% FPR.
- Config. B (eBPF Fast Path Only): Recall drops to 73.1% without Edge-LLM and entropy engine.
- Config. C (Static Rules + LLM, No MTD): Scan resistance drops from 96.8% to 54.2% without MTD.
- Config. D (User-Space Fallback): Throughput drops from 1.49 Mpps to 0.31 Mpps with 19.4% packet loss.

TABLE VII
System Subsystem Ablation Matrix

Configuration	Recall	FPR	Throughput	Pkt Loss
A: Full ASM-Shadhin-AI	98.64%	0.12%	1.49 Mpps	0.00%
B: eBPF Fast-Path Only	73.10%	0.45%	1.51 Mpps	0.00%
C: Without MTD Engine	91.20%	0.18%	1.49 Mpps	0.00%
D: User-Space Fallback	82.40%	3.80%	0.31 Mpps	19.40%

B. Post-Quantum Handshake Overhead

ML-KEM-1024 encapsulation required $48.2\mu\text{s}$ and decapsulation required $59.4\mu\text{s}$, compared to $38.1\mu\text{s}$ for X25519. The marginal $11\text{--}21\mu\text{s}$ overhead occurs only during MTD epoch key initialization (every 30 s) and introduces zero inline latency.

VIII. Defensive Deception & Tarpit Dynamics

A. Reconnaissance Cost Analysis and $1,219\times$ Derivation

The $1,219\times$ reconnaissance-cost factor derives from the following empirical measurement.

Baseline scan time (no MTD): Nmap SYN scan (-sS -p 1-65535 -max-retries 2) against the protected host with a standard Linux firewall: $T_{\text{base}} = 12.4\text{ s}$.

MTD scan time: Same Nmap scan against the MTD-protected host. The tarpit holds TCP connections open with win 0 and 1 byte/s acknowledgment throttling: $T_{\text{MTD}} = 4.2\text{ h} = 15,120\text{ s}$.

Derivation:

$$\text{Cost Multiplier} = \frac{T_{\text{MTD}}}{T_{\text{base}}} = \frac{15,120\text{ s}}{12.4\text{ s}} \approx 1,219 \quad (13)$$

Attacker configuration: Single-host Nmap scan, default retries and timeout settings, full ephemeral port range, one MTD epoch rotation during the scan.

Port-space definition: MTD defends the ephemeral range $[10,000, 65,535]$ (55,536 ports). Well-known ports $[1, 1,023]$ and registered ports $[1,024, 9,999]$ are statically managed and excluded from MTD hopping.

Scope limitations: Adaptive adversaries using reduced per-probe timeouts (e.g., -max-rtt-timeout 200ms) will achieve substantially smaller stall factors. This measurement represents one specific attacker configuration.

B. Adversarial Resource Exhaustion Analysis

A single ASM-Shadhin-AI gateway thread holds up to 65,535 simultaneous TCP connections at win 0 state using 1.2 MB of kernel socket memory, consuming $< 0.1\%$ single-core CPU. Each stalled attacker connection consumes an ephemeral socket, a kernel TCP retransmit timer, and a send buffer on the attacker host. Against a simulated 1,000-node coordinated port scan, ASM-Shadhin-AI simultaneously stalled 62,847 attacker connections (94.3 MB simulated attacker socket memory) while the gateway consumed $< 2\text{ MB}$.

C. Honeypot Port Fingerprinting Resistance

Because $P(s, \tau)$ changes every $\Delta t = 30\text{ s}$ and HMAC-SHA256 PRF output is computationally indistinguishable from uniform random (under standard PRF assumptions), an adversary cannot predict future port assignments without recovering K_{epoch} . The probability of correct epoch port prediction by guessing is $1/55,536 \approx 1.8 \times 10^{-5}$ per epoch.

IX. Limitations & Future Work

Several limitations and open research challenges are acknowledged:

- 1) IPv6 Header Extension Support: The eBPF parsing pipeline supports standard IPv4 and TCP/UDP. Parsing IPv6 extension chains within eBPF verifier instruction limits is an active engineering challenge.
- 2) Hardware SmartNIC Offload: Offloading eBPF bytecode into P4- programmable SmartNIC hardware (e.g., Netronome Agilio CX 40G, NVIDIA BlueField-3 DPU) would enable 40–400 Gbps enforcement with near-zero host CPU utilization.
- 3) Encrypted Protocol Fingerprinting: Incorporating JA4/JA4S TLS and HASSH SSH fingerprinting will enable passive C2 tool identification at the handshake layer.
- 4) Federated Multi-Gateway Coordination: Extending with a federated blacklist synchronization protocol (ML-DSA-65 signed gossip messages) would enable cooperative defense while maintaining data sovereignty.
- 5) LLM Model Updates and Continual Learning: A secure on-device fine-tuning pipeline with differential privacy-preserving gradient updates [9] would address concept drift.
- 6) Evaluation Scale: The Edge-LLM was evaluated on 50 labeled scenarios. Validation on substantially larger labeled datasets with multi-trial evaluations is required to tighten confidence intervals.
- 7) Entropy Detector Generalization: The entropy/jitter detector was validated on simulated C2 traffic; real-world captured C2 samples and diverse benign traffic types require further evaluation.

X. Ethics, Responsible Disclosure, and Reproducibility

A. Ethical Considerations

All evaluations were conducted on a physically isolated, air-gapped testbed with no connectivity to the public internet or production infrastructure. Traffic corpora (CSE-CIC-IDS2018, UNSW-NB15, CTU-13) are publicly available, ethically sourced benchmark datasets. No real user data, private communications, or production traffic was used.

B. Responsible Disclosure

ASM-Shadhin-AI is an open defensive security framework released under the MIT License. All eBPF programs,

configuration files, benchmark scripts, and statistical analysis code are included in the release. No offensive exploit code or novel attack payloads are included.

C. Reproducibility Statement

The following artifacts are committed to the supplementary archive (GitHub repository + Zenodo DOI):

- Source Code: Complete eBPF C source (ebpf/ebpf_filter.c), Python 3.11 daemon, MTD controller, and statistical analysis notebooks.
- Raw Data: $N = 30$ measurement CSV files (testbed/statistical_30_runs_evaluation.csv), model and version information, and LLM evaluation scenario JSONL files.
- Benchmark Scripts: All tcpreplay invocation commands, loop counts, NIC offload configuration scripts, and CPU affinity settings, documented in README.md.
- Statistical Code: Jupyter notebook performing Welch's t-test, Shapiro-Wilk normality tests, Cohen's d , and Wilson confidence intervals (scripts/statistical_analysis.ipynb).
- Hardware: The testbed hardware (Intel i5-4570, 82574L NIC) is commodity off-the-shelf equipment available for $< \$150$ USD, ensuring independent reproducibility.

XI. Conclusion

This paper presented ASM-Shadhin-AI, an open, fully sovereign, line-rate intrusion-mitigation and cognitive-defense architecture. The following conclusions are directly supported by the reported experiments and formal analysis.

In-Kernel Enforcement Performance. The eBPF/XDP pipeline achieved a measured mean per-packet processing latency of $0.124\mu\text{s}$, with an observed XDP-pipeline stage peak of $0.33\mu\text{s}$, under the 10M-packet testbed workload. Compared with Linux Netfilter's mean latency of $26.85\mu\text{s}$, this constitutes a $216\times$ latency reduction ($26.85/0.124 \approx 216.5$). The system sustained 1.492 ± 0.004 Mpps with 0.0000% packet loss under the tested load and 26.2% CPU utilization across $N = 30$ independent trials. These results reflect the specific testbed hardware and workload; generalization requires additional evaluation.

Edge-LLM Semantic Reasoning. Evaluated across 50 labeled threat scenarios (40 malicious/benign, 10 adversarial CFG-stress), the Edge-LLM triage engine achieved 96.0% overall accuracy, 0.967 precision, 0.967 recall, and 0.967 F1-score. All 10 adversarial CFG-stress scenarios produced syntactically valid, executable JSON directives, confirming the CFG constraint's practical effectiveness against the tested inputs. The 50-scenario dataset limits generalizability; evaluation on larger independently curated corpora is required before deployment claims can be made.

Post-Quantum Cryptographic Resilience. Integration of NIST FIPS 203 (ML-KEM-1024) and FIPS 204 (ML-DSA-65) provides IND-CCA2-secure epoch key management

under the MLWE hardness assumption. With per-epoch key erasure, past traffic is protected against SNDL attacks. The PQC overhead of $11\text{--}21\mu\text{s}$ per epoch initialization introduces no inline latency penalty.

Defensive Deception. The combined HMAC-SHA256 epoch-keyed port hopping and tarpit socket exhaustion extended a 65,535-port Nmap SYN scan from 12.4s to 4.2h (a $1,219\times$ increase) under the specific attacker configuration described in Section VIII. Adaptive adversaries using reduced per-probe timeouts will achieve substantially smaller stall factors.

Statistical Validity. Across $N = 30$ independent benchmark trials using Welch's two-sample t-test, ASM-Shadhin-AI's $5.28\times$ throughput improvement over Netfilter and $216\times$ latency reduction are statistically significant ($p < 10^{-15}$, Cohen's $d > 19$), confirming that differences are not attributable to measurement noise. DPDK achieves $2.54\times$ higher raw throughput than ASM-Shadhin-AI but at 100% dedicated CPU utilization.

In aggregate, the experimental results demonstrate that deterministic line-rate defense, locally sovereign cognitive intelligence, and post-quantum cryptographic agility can coexist on commodity x86-64 hardware. Principal open items are: scaling the Edge-LLM evaluation to larger datasets, extending to IPv6, and validating at higher line rates on SmartNIC hardware.

Acknowledgment

The author expresses sincere appreciation to the Department of Computer Science and Engineering at Bangladesh Army University of Science and Technology (BAUST) for providing laboratory resources and testbed computing facilities.

References

- [1] M. Sarhan, S. Layeghifard, N. Moustafa, and M. Gallagher, "Towards a standard feature set for network intrusion detection datasets," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 367–381, 2022.
- [2] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th NDSS Symp.*, 2018, pp. 1–15.
- [3] M. Roesch, "Snort: Lightweight intrusion detection for networks," in *Proc. 13th USENIX LISA Conf.*, 1999, pp. 229–238.
- [4] V. Paxson, "Bro: A system for detecting network intruders in real-time," *Comput. Netw.*, vol. 31, no. 23–24, pp. 2435–2463, 1999.
- [5] S. Miano, M. Bertrone, F. Risso, M. Tumolo, and M. Bernal, "Creating complex network services with eBPF: Experience and lessons learned," in *Proc. IEEE HPCC*, 2018, pp. 1–8.
- [6] T. Høiland-Jørgensen et al., "The eXpress data path: Fast programmable packet processing in the operating system kernel," in *Proc. 14th ACM CoNEXT*, 2018, pp. 54–66.
- [7] S. Miano, R. Doriguzzi-Corin, F. Risso, D. Siracusa, and R. Sompalle, "Introducing SmartNIC support in BEBA," in *Proc. IEEE NetSoft*, 2018, pp. 1–9.
- [8] M. Ring et al., "A survey of network-based intrusion detection data sets," *Comput. Secur.*, vol. 86, pp. 147–167, 2019.
- [9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. NeurIPS*, vol. 36, 2023, pp. 10088–10115.
- [10] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS PUB 203, Aug. 2024.

- [11] National Institute of Standards and Technology, “Module-Lattice-Based Digital Signature Standard,” FIPS PUB 204, Aug. 2024.
- [12] S. Jajodia et al., Eds., *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*. New York: Springer, 2011.
- [13] Open Information Security Foundation, “Suricata User Guide Release 7.0.2,” Tech. Rep., 2024. [Online]. Available: <https://suricata.io>
- [14] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. ICISSP*, 2018, pp. 108–116.
- [15] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proc. IEEE MilCIS*, 2015, pp. 1–6.
- [16] S. Garcia, M. Grill, J. Stiborek, and P. Zunino, “An empirical comparison of botnet detection methods,” *Comput. Secur.*, vol. 45, pp. 100–123, 2014.
- [17] B. Anderson and D. McGrew, “Identifying encrypted malware traffic with contextual flow data,” in *Proc. ACM IH&MMSec*, 2016, pp. 35–46.
- [18] F. Tegeler, X. Fu, G. Vigna, and C. Kruegel, “BotFinder: Finding bots in network traffic without deep packet inspection,” in *Proc. ACM CoNEXT*, 2012, pp. 349–360.
- [19] G. Bertoli, L. Verderame, and A. Merlo, “eBPF for cyber security: A comprehensive survey,” *IEEE Access*, vol. 11, pp. 75892–75916, 2023.
- [20] M. Alweshah, S. Khadem, and M. Al-Omari, “Intrusion detection system using machine learning algorithms: A survey,” *J. Netw. Comput. Appl.*, vol. 165, p. 102712, 2020.
- [21] Q. Zhao, J. Chen, and D. Guan, “GRAPHIDS: A network anomaly intrusion detection system based on graph neural network,” *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 4, pp. 4215–4228, 2023.
- [22] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, “Post-quantum key exchange—a new hope,” in *Proc. 25th USENIX Security Symp.*, 2016, pp. 327–343.
- [23] M. Atighetchi, P. Pal, F. Webber, and C. Jones, “Adaptive use of network-centric mechanisms in cyber-defense,” in *Proc. IEEE ISORC*, 2003, pp. 183–192.
- [24] P. Sharma and R. Arora, “Evaluating post-quantum cryptographic primitives in lightweight network protocols,” *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16201–16212, 2023.
- [25] P. Wang, F. Ye, X. Chen, and Y. Qian, “Datanet: Deep learning based encrypted network traffic classification in SDN home gateway,” *IEEE Access*, vol. 6, pp. 55380–55391, 2018.
- [26] M. A. S. Vieira et al., “Fast packet processing with eBPF and XDP: Security on the edge,” *IEEE Commun. Surv. Tuts.*, vol. 22, no. 4, pp. 2526–2544, 2020.
- [27] J. Zheng, A. Chowdhury, and D. Guan, “Moving target defense for network security: A survey,” *IEEE Commun. Surv. Tuts.*, vol. 24, no. 3, pp. 1800–1835, 2022.
- [28] D. J. Bernstein and T. Lange, “Post-quantum cryptography,” *Nature*, vol. 549, pp. 188–194, 2017.
- [29] O. Alrawi, C. Lever, M. Antonakakis, and F. Monrose, “SoK: Security evaluation of home IoT devices,” in *Proc. IEEE S&P*, 2019, pp. 1052–1069.
- [30] P. Bosshart et al., “P4: Programming protocol-independent packet processors,” *ACM SIGCOMM CCR*, vol. 44, no. 3, pp. 87–95, 2014.
- [31] H. Goudarzi, M. Naderi, and M. S. Sadoghi, “Hardware implementations of NIST post-quantum cryptography standards: A survey,” *IEEE Access*, vol. 11, pp. 42150–42168, 2023.
- [32] H. Choi, M. Kim, and J. Lee, “LLM-based autonomous agent for cyber threat hunting and defense,” *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 3, pp. 1420–1435, 2024.
- [33] G. Cormode and S. Muthukrishnan, “An improved data stream summary: The count-min sketch and its applications,” *J. Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.
- [34] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [35] R. K. Jain et al., “A quantitative measure of fairness for resource allocation in shared systems,” *DEC Tech. Rep. TR-301*, 1984.



A S M Hossain Mahmud (Shadhin) is a full-time undergraduate student pursuing the B.Sc. degree in Computer Science and Engineering at Bangladesh Army University of Science and Technology (BAUST), Saidpur 5310, Bangladesh. His primary research interests include kernel-space high-throughput packet processing using eBPF/XDP, post-quantum cryptographic implementations (NIST FIPS 203 ML-KEM-1024 and FIPS 204 ML-DSA-65), sovereign local AI architectures for automated threat reasoning, proactive moving target defense (HMAC-SHA256 port hopping), and active cyber deception. He is the lead architect and developer of the ASM-Shadhin-AI framework.